DESARROLLO FULL STACK · UNIDAD 2

Frameworks de back-end y CRUD con base de datos

Plataformas de desarrollo · API REST con Express + SQLite, de la petición HTTP a la tabla

Facultad de Ingeniería de Sistemas · Sesión teórico-práctica de 90 minutos

Docente: Raky Carbajal Ramos

RUTA DE LA SESIÓN

90 minutos en cinco momentos

Momento	Minutos	Qué hacemos
Inicio	0 – 10	Pregunta detonante y logro
Utilidad	10 – 15	El back-end en apps que usas a diario
Transformación	15 – 55	Frameworks, plataformas, CRUD ↔ HTTP ↔ SQL y ejemplo en vivo
Práctica	55 – 85	Laboratorio: tu propia API CRUD
Cierre	85 – 90	Puente al front-end y reflexión

LOGRO DE LA SESIÓN

Implementas una API REST con operaciones CRUD conectada a una base de datos y la pruebas con un cliente HTTP.

INICIO · PREGUNTA DETONANTE

Presionas «Guardar» en una app. ¿Qué viaja, a dónde y quién lo guarda?

1 · ¿Qué datos salen del navegador y en qué formato?

2 · ¿Qué programa los recibe y los valida?

3 · ¿Dónde quedan cuando cierras la app?

UTILIDAD

Cada acción en pantalla es una operación en la base de datos

Banca móvil

Consultar saldo es leer. Transferir es actualizar dos cuentas a la vez.

Delivery

Hacer un pedido es crear. Seguirlo en el mapa es leer su estado una y otra vez.

Matrícula

Inscribirte en un curso es crear. Retirarte es eliminar. Cambiar de sección es actualizar.

Crear, Leer, Actualizar, Eliminar: el CRUD está detrás de casi todo lo que hace un usuario.

TRANSFORMACIÓN · CONCEPTO

¿Qué es un framework de back-end?

Un conjunto de herramientas y convenciones para construir el **servidor**: recibe peticiones HTTP, aplica reglas de negocio, habla con la base de datos y responde.

Sin framework escribirías a mano el manejo de sockets, la lectura del cuerpo de cada petición y el ruteo. El framework resuelve lo repetitivo; tú te concentras en la lógica.

Enrutamiento

Asocia método + URL con una función.

Middleware

Pasos previos: leer JSON, CORS, registros.

Acceso a datos

Consultas SQL directas o un ORM.

Seguridad

Validación, autenticación, permisos.

TRANSFORMACIÓN · PANORAMA

Frameworks de back-end más usados

Framework	Lenguaje	Estilo	Dónde lo verás
Express	JavaScript (Node.js)	Minimalista: tú eliges las piezas	APIs ligeras, prototipos
NestJS	TypeScript (Node.js)	Estructurado en módulos	Proyectos Node grandes
Spring Boot	Java	Completo, con anotaciones	Banca, sector corporativo
Django / FastAPI	Python	Completo / ligero y tipado	Datos, IA, paneles admin
Laravel	PHP	Completo, ORM incluido	Web tradicional, e-commerce
ASP.NET Core	C#	Completo, ecosistema Microsoft	Empresas con stack .NET

Hoy: Express. Poco código y todo el flujo a la vista. Los conceptos (rutas, middleware, CRUD) se trasladan a Spring Boot, que ya trabajamos.

TRANSFORMACIÓN · PLATAFORMAS DE DESARROLLO

Lo que necesitas instalado para trabajar

Entorno de ejecución

Node.js LTS · JDK · Python

Gestor de paquetes

npm · Maven · pip

Editor o IDE

VS Code · IntelliJ · NetBeans

Cliente HTTP

REST Client · Thunder Client · Postman

Base de datos

SQLite → MySQL · PostgreSQL

Control de versiones

Git + GitHub

En **negrita**, lo que usaremos hoy. SQLite guarda la base en un solo archivo: cero instalación de servidor, ideal para aprender.

TRANSFORMACIÓN · ARQUITECTURA

El viaje de una petición, de ida y vuelta



Ida → método HTTP + URL + cuerpo JSON. Express elige la ruta y la consulta llega a la tabla.

← Vuelta filas SQL → objeto JavaScript → JSON + código de estado (200, 201, 404…).

TRANSFORMACIÓN · LA TABLA CLAVE

CRUD ↔ HTTP ↔ SQL

Operación	Método	Ruta	SQL	Éxito
Crear	POST	/api/libros	INSERT INTO …	201 Created
Leer todos	GET	/api/libros	SELECT * FROM …	200 OK
Leer uno	GET	/api/libros/:id	SELECT … WHERE id	200 OK
Actualizar	PUT	/api/libros/:id	UPDATE … WHERE id	200 OK
Eliminar	DELETE	/api/libros/:id	DELETE … WHERE id	204 No Content

La URL nombra el **recurso** (libros, en plural); el método dice la **acción**. Nunca /api/crearLibro ni /api/borrarLibro.

TRANSFORMACIÓN · CÓDIGOS DE ESTADO

La API responde con un número que significa algo

200

OK · éxito: lectura o actualización

201

Created · éxito: recurso creado

204

No Content · éxito sin cuerpo: eliminado

400

Bad Request · error del cliente: datos inválidos

404

Not Found · error del cliente: no existe

500

Server Error · error del servidor: falló el código

Regla rápida: **2xx** salió bien · **4xx** el cliente envió algo mal · **5xx** el servidor falló.

EJEMPLO EN VIVO · PASO 1

Proyecto «api-biblioteca»: crear y preparar

TERMINAL

```
$ mkdir api-biblioteca
$ cd api-biblioteca
$ npm init -y
$ npm i express better-sqlite3 cors
$ npm run dev
API escuchando en http://localhost:3000
```

ESTRUCTURA

```
api-biblioteca/
├── app.js          ← rutas CRUD
├── db.js           ← conexión + tabla
├── peticiones.http ← pruebas
├── package.json
└── biblioteca.db   ← se crea sola
```

Tres dependencias: **express** (framework), **better-sqlite3** (driver de la base), **cors** (permitir al front-end).

EJEMPLO EN VIVO · PASO 2 · DB.JS

Conectar la base de datos y crear la tabla

```
// db.js — conexión a SQLite y creación de la tabla
const Database = require('better-sqlite3');

const db = new Database('biblioteca.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS libros (
    id     INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT    NOT NULL,
    autor  TEXT    NOT NULL,
    anio   INTEGER,
    stock  INTEGER NOT NULL DEFAULT 0
  )`);

module.exports = db;
```

1 · Abre el archivo de la base; si no existe, lo crea.

2 · **IF NOT EXISTS**: puedes reiniciar el servidor sin borrar datos.

3 · **AUTOINCREMENT**: la base asigna el id, no el cliente.

4 · Exportamos una sola conexión para toda la app.

EJEMPLO EN VIVO · PASO 3 · APP.JS

Configurar Express y la ruta READ

```
const express = require('express');
const cors = require('cors');
const db = require('./db');
const app = express();
app.use(cors());          // permite llamadas del front
app.use(express.json());  // lee el JSON del body

// READ: listar (filtro opcional ?autor=)
app.get('/api/libros', (req, res) => {
  const { autor } = req.query;
  const filas = autor
    ? db.prepare('SELECT * FROM libros WHERE autor LIKE ?')
        .all(`%${autor}%`)
    : db.prepare('SELECT * FROM libros').all();
  res.json(filas);
});
```

req trae lo que envió el cliente; **res** es lo que respondemos.

req.query lee lo que va después del «?» en la URL.

.all() devuelve un arreglo de filas; **res.json** lo envía como JSON con estado 200.

EJEMPLO EN VIVO · PASO 4 · APP.JS

CREATE: validar y luego insertar

```
// Validación simple del cuerpo
function validar(body) {
  if (!body.titulo || !body.autor)
    return 'titulo y autor son obligatorios';
  if (body.stock !== undefined
      && body.stock < 0)
    return 'stock no puede ser negativo';
  return null;
}
```

```
// CREATE: registrar un libro
app.post('/api/libros', (req, res) => {
  const error = validar(req.body);
  if (error) return res.status(400).json({ error });
  const { titulo, autor, anio = null, stock = 0 }
    = req.body;
  const r = db.prepare(
    'INSERT INTO libros (titulo, autor, anio, stock)' +
    ' VALUES (?, ?, ?, ?)'
  ).run(titulo, autor, anio, stock);
  res.status(201).json({
    id: r.lastInsertRowid, titulo, autor, anio, stock
  });
});
```

Validamos **antes** de tocar la base: si faltan datos respondemos **400** y el INSERT nunca se ejecuta.

RESULTADO · EJECUCIÓN REAL

POST: un registro válido y uno inválido

POST /api/libros

```
{ "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967, "stock": 5 }
```

201 Created · éxito

```
{"id":1,"titulo":"Cien años de soledad",
 "autor":"Gabriel García Márquez",
 "anio":1967,"stock":5}
```

POST /api/libros

```
{ "titulo": "Sin autor" }
```

400 Bad Request · error

```
{"error":"titulo y autor son obligatorios"}
```

Registramos dos libros más (Vargas Llosa y Arguedas): la base les asignó **id 2** e **id 3**.

EJEMPLO EN VIVO · PASO 5 · APP.JS

Rutas con :id — leer uno y eliminar

```
// READ: obtener uno por id
app.get('/api/libros/:id', (req, res) => {
  const libro = db.prepare('SELECT * FROM libros WHERE id = ?')
    .get(req.params.id);
  if (!libro)
    return res.status(404).json({ error: 'Libro no encontrado' });
  res.json(libro);
});

// DELETE: eliminar
app.delete('/api/libros/:id', (req, res) => {
  const r = db.prepare('DELETE FROM libros WHERE id = ?')
    .run(req.params.id);
  if (r.changes === 0)
    return res.status(404).json({ error: 'Libro no encontrado' });
  res.status(204).end();
});
```

req.params.id toma el valor de la URL: en /api/libros/2 vale «2».

.get() devuelve una fila o *undefined*; si no hay fila, respondemos **404**.

r.changes cuenta las filas afectadas: 0 significa que el id no existía.

RESULTADO · EJECUCIÓN REAL

GET: listar, buscar por id y filtrar

GET /api/libros — 200 OK


```
[
 {"id":1,"titulo":"Cien años de soledad","autor":"Gabriel García Márquez","anio":1967,"stock":5},
 {"id":2,"titulo":"La ciudad y los perros","autor":"Mario Vargas Llosa","anio":1963,"stock":3},
 {"id":3,"titulo":"Los ríos profundos","autor":"José María Arguedas","anio":1958,"stock":2}
]
```


GET /api/libros/2 — 200 OK


```
{"id":2,"titulo":"La ciudad y los perros","autor":"Mario Vargas Llosa","anio":1963,"stock":3}
```


GET /api/libros?autor=Vargas — 200 OK


```
[{"id":2,"titulo":"La ciudad y los perros","autor":"Mario Vargas Llosa","anio":1963,"stock":3}]
```

EJEMPLO EN VIVO · PASO 6 · APP.JS

UPDATE y levantar el servidor

```
// UPDATE: actualizar
app.put('/api/libros/:id', (req, res) => {
  const error = validar(req.body);
  if (error) return res.status(400).json({ error });
  const { titulo, autor, anio = null, stock = 0 } = req.body;
  const r = db.prepare(
    'UPDATE libros SET titulo=?, autor=?, anio=?, stock=?' +
    ' WHERE id=?'
  ).run(titulo, autor, anio, stock, req.params.id);
  if (r.changes === 0)
    return res.status(404).json({ error: 'Libro no encontrado' });
  res.json({ id: Number(req.params.id), titulo, autor, anio, stock });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () =>
  console.log(`API escuchando en http://localhost:${PORT}`));
```

PUT combina lo aprendido: **validar** como en POST y **:id + changes** como en DELETE.

Reutilizamos **validar()**: una sola regla para crear y actualizar.

process.env.PORT permite que el hosting elija el puerto al desplegar.

RESULTADO · EJECUCIÓN REAL

PUT, DELETE y el caso «no existe»

PUT /api/libros/2 · stock de 3 a 10 — **200 OK**

{"id":2,"titulo":"La ciudad y los perros","autor":"Mario Vargas Llosa","anio":1963,"stock":10}

DELETE /api/libros/3 — **204 No Content**

(respuesta sin cuerpo)

GET /api/libros/3 · después de borrarlo — **404 Not Found · error**

{"error":"Libro no encontrado"}

RESULTADO · LA TABLA DESPUÉS DE TODO

Lo que quedó guardado en biblioteca.db

```
sqlite> SELECT * FROM libros;
```

id	titulo	autor	anio	stock
1	Cien años de soledad	Gabriel García Márquez	1967	5
2	La ciudad y los perros	Mario Vargas Llosa	1963	10

id 2: stock pasó de 3 a **10** (PUT).

id 3: ya no está (DELETE).

Si reinicias el servidor, los datos **persisten**: viven en el archivo, no en memoria.

BUENAS PRÁCTICAS

Consultas parametrizadas: nunca concatenes

Inseguro

```
db.prepare(`SELECT * FROM libros WHERE id = ${req.params.id}`)
```

Seguro

```
db.prepare('SELECT * FROM libros WHERE id = ?').get(req.params.id)
```

Valida todo

Nunca confíes en lo que envía el cliente.

Código correcto

201 al crear, 404 si no existe, 400 si es inválido.

Variables de entorno

Puerto y credenciales fuera del código.

Capas separadas

Rutas, lógica y datos en archivos distintos al crecer.

PRÁCTICA · LABORATORIO DE 30 MINUTOS

Reto: API REST de cursos de la facultad

cursos(id, codigo, nombre, creditos, ciclo)

1. Proyecto Express + SQLite con las 5 rutas CRUD en /api/cursos.
2. codigo y nombre obligatorios; creditos entre 1 y 6 → si no, 400.
3. 404 cuando el id no existe al leer, actualizar o eliminar.
4. Filtro opcional GET /api/cursos?ciclo=5.
5. Solo consultas parametrizadas con «?».
6. Evidencia: peticiones.http y captura de cada respuesta.

Min 0 – 10
Proyecto, tabla, GET y POST

Min 10 – 20
GET por id, PUT y DELETE

Min 20 – 30
Validaciones, filtro y evidencias

PRÁCTICA · CÓMO SE EVALÚA

Rúbrica del laboratorio (20 puntos)

Criterio	Logrado cuando…	Puntos
Rutas CRUD	Las 5 rutas responden y modifican la tabla	8
Validación y estados	400, 404, 201 y 204 donde corresponde	5
Seguridad	Todas las consultas usan parámetros «?»	3
Filtro por ciclo	?ciclo= devuelve solo los cursos del ciclo	2
Evidencias	peticiones.http + capturas de cada respuesta	2
Total		20

CIERRE · PUENTE CON LA UNIDAD

Tu front-end en React consumirá esta API

```
// React + Vite: listar al cargar el componente
useEffect(() => {
  fetch('http://localhost:3000/api/libros')
    .then(res => res.json())
    .then(setLibros);
}, []);

// Enviar un formulario: crear un libro
await fetch('http://localhost:3000/api/libros', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(nuevoLibro),
});
```

El front (localhost:5173) y la API (localhost:3000) son **orígenes distintos**: por eso activamos **cors()**.

La tabla CRUD ↔ HTTP es el contrato entre ambos equipos.

Próxima sesión: consumo de servicios y formularios.

CIERRE

Tres ideas para llevarte

01

Un framework de back-end resuelve rutas, middleware y acceso a datos.

02

CRUD se traduce a POST, GET, PUT, DELETE y a INSERT, SELECT, UPDATE, DELETE.

03

Validar, responder el código correcto y parametrizar el SQL no es opcional.

REFLEXIONA

¿Qué parte del viaje petición → SQL → respuesta te costó más? · ¿Qué cambiarías para usar MySQL en lugar de SQLite? · Vuelve a la pregunta inicial: ¿ahora puedes responderla completa?